

Puissance 4

Technical Report

Achagui Aymen

ENSEEIHIT — Computer Engineering Student

Stack	Java 17, Spring Boot 3, PostgreSQL
DevOps	Docker, Kubernetes (minikube/EKS), GitHub Actions
AI	Gemini 3.5 Flash (PR Reviewer prototype)

Contents

1	Project Overview	2
1.1	Architecture Overview	2
2	Application Features	2
2.1	Authentication	2
2.2	Game Logic	3
2.3	Real-Time Communication	3
2.4	Frontend	3
3	Containerization	3
3.1	Multi-Stage Dockerfile	3
3.2	Docker Compose	5
4	Kubernetes Deployment	5
4.1	Deployment Strategy	5
4.2	Horizontal Pod Autoscaler	6
5	CI/CD Pipeline	6
6	AI-Powered PR Reviewer	7
6.1	Motivation	7
6.2	Architecture	7
6.3	Implementation	8
6.4	Example Output	9
6.5	CI/CD Integration	9
7	Summary	10

Project Overview

Puissance 4 is a cloud-native real-time multiplayer Connect Four application built as a full-stack engineering project. The goal was not only to implement the game logic but to architect, containerize, and deploy a production-grade backend with a complete CI/CD pipeline.

The application exposes a REST API for authentication, game management, and rankings, complemented by a WebSocket layer for real-time gameplay. The entire stack is containerized with Docker and deployed on Kubernetes, with an AI-powered pull request reviewer integrated into the development workflow.

Architecture Overview

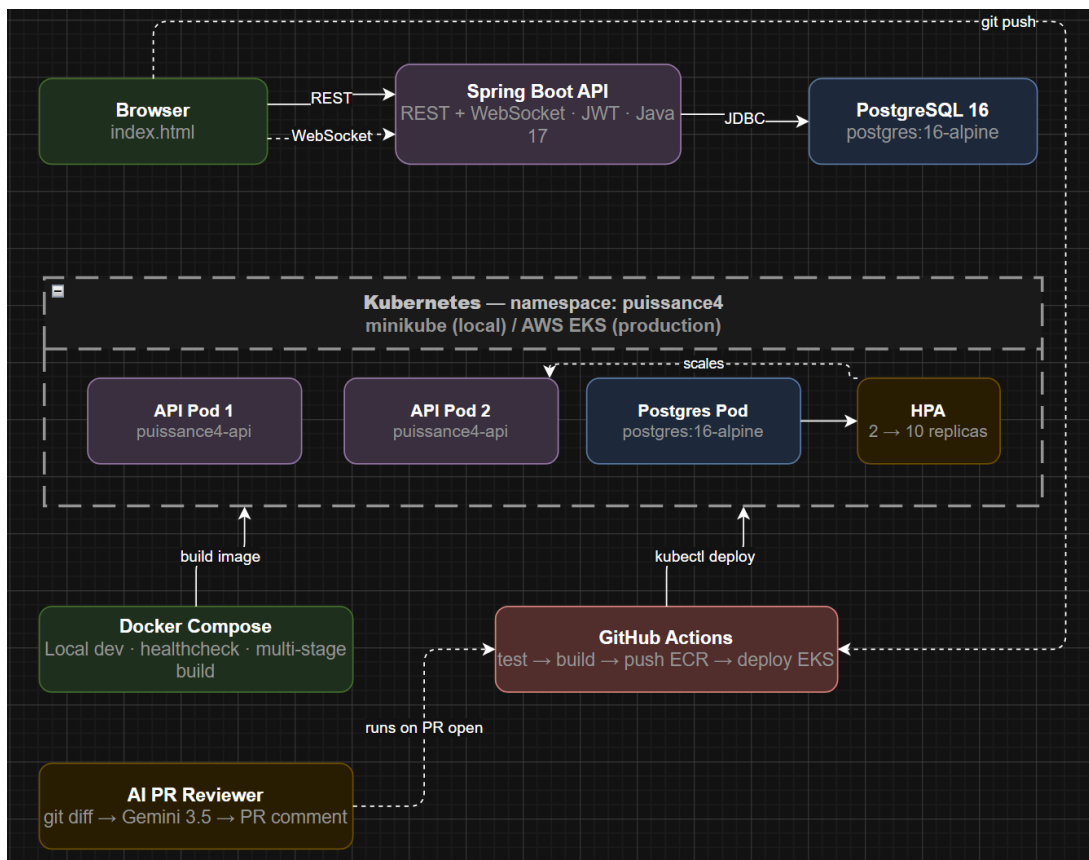


Figure 1: Application architecture

Application Features

Authentication

JWT-based authentication. Users register and receive a signed token, which is validated on every subsequent REST request and WebSocket message.

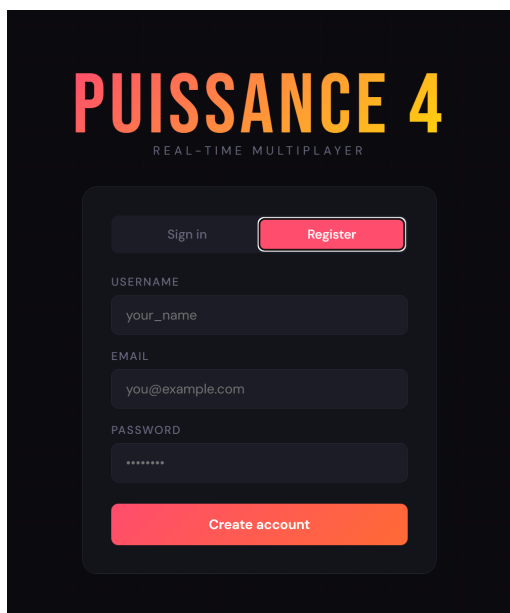
Game Logic

The board is stored as a serialized string: 0000000,0000000,... — 6 rows of 7 columns. Token placement applies gravity (scanning upward from the bottom row). Win detection checks all four directions after every move: horizontal, vertical, diagonal, and anti-diagonal.

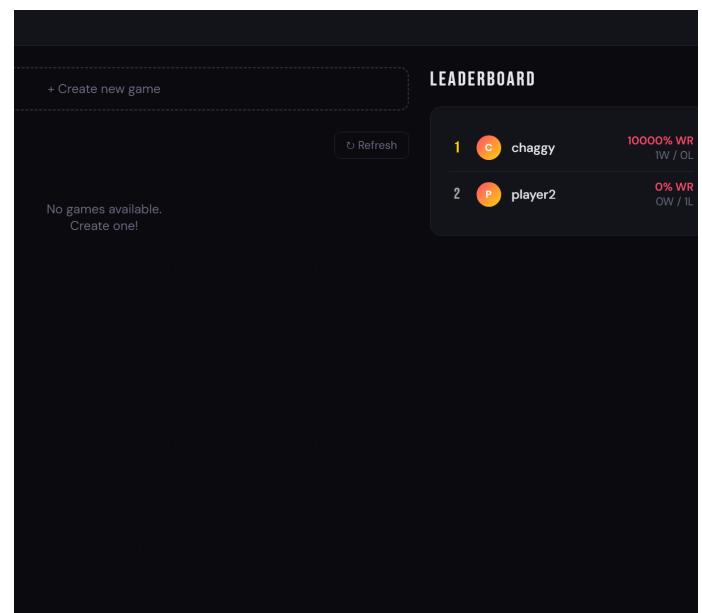
Real-Time Communication

WebSocket with STOMP protocol. Moves, chat messages, and game events are broadcast to both players via `/topic/game/{gameId}` without polling.

Frontend



(a) Login / Register



(b) Game Lobby

Figure 2: Frontend — authentication and lobby

Containerization

Multi-Stage Dockerfile

A multi-stage build keeps the final image minimal and secure. The build stage compiles the application with Maven; the runtime stage uses a minimal Alpine JRE image and runs as a non-root user.

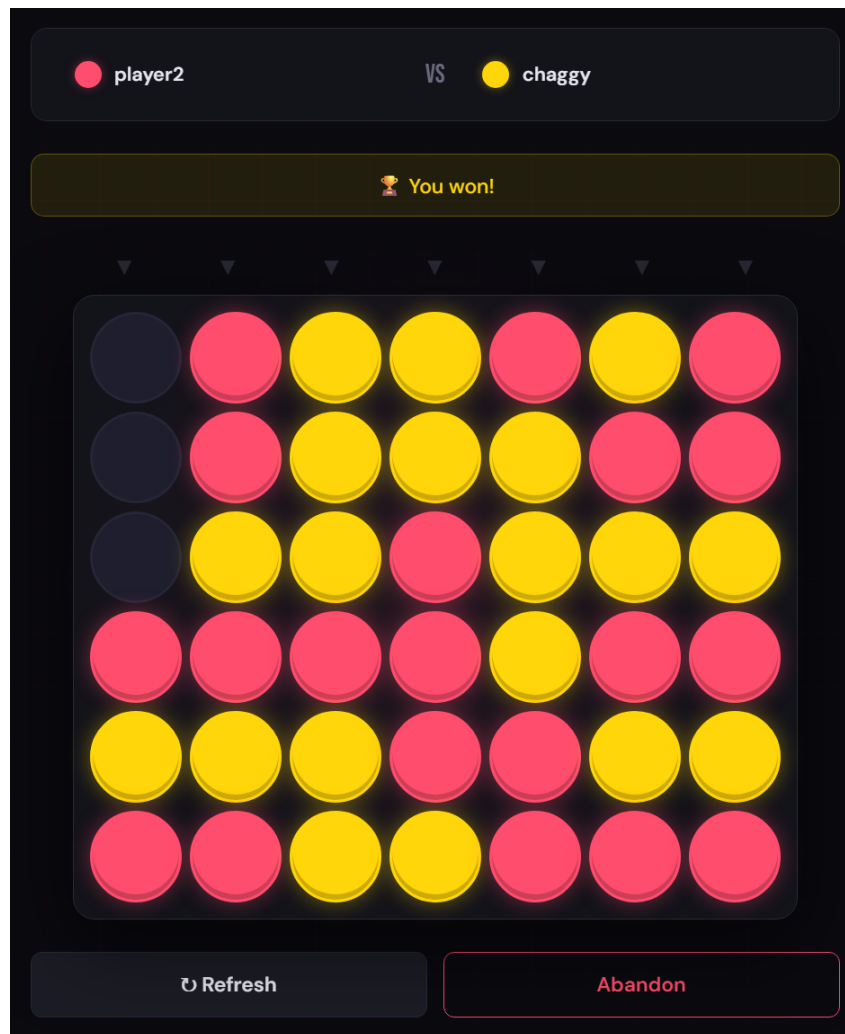


Figure 3: Game board — real-time play with turn indicators

```

1 # Stage 1: Build
2 FROM maven:3.8.7-eclipse-temurin-17 AS builder
3 WORKDIR /app
4 COPY pom.xml .
5 RUN mvn dependency:go-offline -q
6 COPY src ./src
7 RUN mvn package -DskipTests -q
8
9 # Stage 2: Runtime (minimal image)
10 FROM eclipse-temurin:17-jre-alpine
11 RUN addgroup -S appgroup && adduser -S appuser -G appgroup
12 WORKDIR /app
13 COPY --from=builder /app/target/*.jar app.jar
14 USER appuser # non-root for security
15 EXPOSE 8080
16 ENTRYPOINT ["java", "-jar", "app.jar"]

```

Listing 1: Multi-stage Dockerfile

Docker Compose

Local development stack with a PostgreSQL healthcheck — the API only starts once the database is fully ready.

```
1 services:
2   db:
3     image: postgres:16-alpine
4     healthcheck:
5       test: ["CMD-SHELL", "pg_isready -U postgres"]
6
7   api:
8     build: .
9     depends_on:
10      db:
11        condition: service_healthy    # wait for DB before
                                         starting
```

Listing 2: docker-compose.yml (simplified)

Kubernetes Deployment

The application is deployed on Kubernetes (locally via minikube, production-ready for AWS EKS). The deployment uses a rolling update strategy to ensure zero downtime.

Deployment Strategy

```
1 spec:
2   replicas: 2
3   strategy:
4     type: RollingUpdate
5     rollingUpdate:
6       maxSurge: 1          # allow 1 extra pod during update
7       maxUnavailable: 0    # never kill old pod before new one is
                              ready
8   template:
9     spec:
10      containers:
11        - name: puissance4-api
12          livenessProbe:
13            httpGet:
14              path: /api/auth/health
15              port: 8080
16          readinessProbe:
17            httpGet:
18              path: /api/auth/health
19              port: 8080
```

Listing 3: deployment.yaml — rolling update configuration

Horizontal Pod Autoscaler

The HPA automatically scales the deployment from 2 to 10 replicas based on CPU and memory utilization.

```
1 spec:
2   scaleTargetRef:
3     kind: Deployment
4     name: puissance4-api
5   minReplicas: 2
6   maxReplicas: 10
7   metrics:
8     - type: Resource
9       resource:
10        name: cpu
11        target:
12          averageUtilization: 60
13     - type: Resource
14       resource:
15        name: memory
16        target:
17          averageUtilization: 70
```

Listing 4: hpa.yaml

```
chaggyneutron@CHAGGYYYY:~/puissance4$ kubectl get pods -n puissance4
NAME                                READY   STATUS    RESTARTS   AGE
postgres-5cf8d69b86-dfttw          1/1     Running   0           23h
puissance4-api-86948dc664-68phk     1/1     Running   0           137m
puissance4-api-86948dc664-fjhwq     1/1     Running   0           135m
```

Figure 4: Kubernetes pods running locally on minikube

CI/CD Pipeline

The GitHub Actions pipeline runs on every push to `main` and every pull request. It consists of three jobs: test, build & push to ECR, and deploy to EKS.

```

1 jobs:
2   test:                                # runs on every push and PR
3     steps:
4       - name: Run tests
5         run: ./mvnw test
6
7   build-push:                          # only on main branch
8     needs: test
9     steps:
10      - name: Build and push to ECR
11        run: |
12          docker build -t $REGISTRY/$ECR_REPO:${{ github.sha }} .
13          docker push $REGISTRY/$ECR_REPO:${{ github.sha }}
14
15   deploy:                              # only after build-push succeeds
16     needs: build-push
17     steps:
18       - name: Rolling update on EKS
19         run: |
20           kubectl set image deployment/puissance4-api \
21             api=$IMAGE -n puissance4
22       - name: Verify rollout
23         run: kubectl rollout status deployment/puissance4-api --
              timeout=5m

```

Listing 5: ci-cd.yml — pipeline structure

Each image is tagged with the Git commit SHA for full traceability. Rolling back to a previous version is as simple as:

```
kubectl set image deployment/puissance4-api api=registry/repo:<previous-sha>
```

AI-Powered PR Reviewer

Motivation

One of the key DevOps challenges is maintaining code quality at scale. This prototype automates pull request reviews by sending the git diff to an LLM and posting the feedback as a GitHub comment — directly in the developer’s workflow.

Architecture

Open PR → Fetch diff (GitHub API) → Send to Gemini → Post comment on PR

Implementation

```

1 def get_pr_diff(pr_number):
2     """Fetch the raw diff of a PR via GitHub API."""
3     url = f"https://api.github.com/repos/{REPO}/pulls/{pr_number}"
4     headers = {
5         "Authorization": f"token {GITHUB_TOKEN}",
6         "Accept": "application/vnd.github.v3.diff"
7     }
8     return requests.get(url, headers=headers).text
9
10 def review_with_gemini(diff):
11     """Send diff to Gemini and get a structured code review."""
12     prompt = f"""You are a senior code reviewer. Analyze this git
13         diff and provide:
14     1. Security issues
15     2. Bugs
16     3. Bad practices
17     4. Summary of changes
18
19     Git diff:
20     {diff}
21     """
22     response = requests.post(
23         f"https://generativelanguage.googleapis.com/v1beta/models/
24         /"
25         f"gemini-3.5-flash:generateContent?key={GEMINI_API_KEY}",
26         headers={"Content-Type": "application/json"},
27         json={"contents": [{"parts": [{"text": prompt}]}]}
28     )
29     data = response.json()
30     if "error" in data:
31         return f"Gemini error: {data['error']['message']}"
32     return data["candidates"][0]["content"]["parts"][0]["text"]
33
34 def post_pr_comment(pr_number, comment):
35     """Post the review as a comment on the GitHub PR."""
36     url = f"https://api.github.com/repos/{REPO}/issues/{pr_number}
37         /comments"
38     body = f"## AI Code Review\n\n{comment}\n\n*Powered by Gemini
39         3.5 Flash*"
40     response = requests.post(url,
41         headers={"Authorization": f"token {GITHUB_TOKEN}"},
42         json={"body": body})
43     return response.status_code == 201

```

Listing 6: ai_reviewer.py — core logic

```

chaggyneutron@CHAGGYYYY:~/puissance4$ python3 ./ai_reviewer.py 1
Fetching diff for PR #1...
Sending to Gemini for review...
Posting comment on PR...
✅ Review posted on PR #1

--- Review ---
### 1. Security Issues
* **No hardcoded secrets found**: The PR correctly uses environment variables ('GEMINI_API_KEY', 'GITHUB_TOKEN', and GitHub Secrets) instead of hardcoding sensitive credentials.
* **Missing HTTP timeouts**: The 'requests' calls in 'ai_reviewer.py' do not specify a 'timeout' parameter. While not a direct exploit, this can lead to Denial of Service (DoS) in the runner if the external APIs hang indefinitely. Always set a timeout (e.g., 'timeout=10').

---

### 2. Bugs
* **Critical Typo in Workflow**: In '.github/workflows/ci-cd.yml', 'aws-access-key-id' was renamed to 'aws-access-key-i'. This typo will cause the AWS authentication step to fail, breaking your CD pipeline.
* **Invalid Gemini Model**: In 'ai_reviewer.py', the model specified is 'gemini-3.5-flash'. There is currently no "3.5" version of Gemini (use 'gemini-1.5-flash' or 'gemini-2.0-flash' instead). This API call will fail with a 404 error.
* **Fragile JSON Parsing**: In 'ai_reviewer.py' ('review_with_gemini'):
  ```python
 return data["candidates"][0]["content"]["parts"][0]["text"]
  ```
  If the Gemini API responds with a 200 OK but blocked content (e.g., safety filters), the 'candidates' array may be empty or structured differently, throwing an unhandled 'IndexError' or 'KeyError'.

---

### 3. Bad Practices
* **Hardcoded Repository Path**: In 'ai_reviewer.py', the 'REPO' variable is hardcoded to 'chaggyneutron/puissance4'. To make this script modular and reusable, fetch this dynamically from the default GitHub Environment variable:
  ```python
 REPO = os.getenv("GITHUB_REPOSITORY")
  ```
* **No HTTP Status Code Verification**: The function 'get_pr_diff' returns 'response.text' directly without checking if 'response.status_code == 200'. If GitHub returns a '401 Unauthorized' or '404 Not Found', the script will feed the error page/JSON into Gemini as if it were code.

---

### 4. Summary
This PR introduces a Python script ('ai_reviewer.py') designed to automate pull request reviews using the Gemini API and post them as GitHub comments. Additionally, it adjusts the test execution behavior in CI/CD and attempts to configure AWS credentials, though the latter contains a critical typo that will break the build.

```

Figure 5: AI-generated review posted as a GitHub PR comment

Example Output

CI/CD Integration

The reviewer can be integrated as a GitHub Actions job that triggers automatically on every pull request:

```

1  ai-review:
2    runs-on: ubuntu-latest
3    if: github.event_name == 'pull_request'
4    steps:
5      - uses: actions/checkout@v4
6      - name: Run AI reviewer
7        env:
8          GEMINI_API_KEY: ${ secrets.GEMINI_API_KEY }
9          GITHUB_TOKEN:   ${ secrets.GITHUB_TOKEN }
10     run: python3 ai_reviewer.py ${ github.event.pull_request
        .number }

```

Listing 7: GitHub Actions job for automatic AI review

| Component | Technology | Purpose |
|--------------------------|---------------------------|-------------------------------|
| Backend API | Spring Boot 3, Java 17 | REST + WebSocket |
| Database | PostgreSQL 16 | Persistence |
| Authentication | JWT | Stateless auth |
| Containerization | Docker (multi-stage) | Portable deployment |
| Local orchestration | minikube + kubectl | Kubernetes dev environment |
| Production orchestration | AWS EKS + HPA | Auto-scaling deployment |
| CI/CD | GitHub Actions | Automated test, build, deploy |
| AI Reviewer | Python + Gemini 3.5 Flash | Automated PR code review |
| Frontend | HTML/CSS/JS | Single-file UI |

Table 1: Full technology stack

Summary

The project demonstrates the full DevOps lifecycle — from local development with Docker Compose, through containerization and Kubernetes orchestration, to a complete CI/CD pipeline with an AI-augmented code review step integrated directly into the pull request workflow.